POLYTECHNIQUE MONTRÉAL

Questionnaire examen final

LOG2990

Sigle du cours

15.5

Identification de l'étudiant(e)		
Nom : DEFAICHTERE	Prénom : ANTHONY	
Signature :	Matricule : 2004221	Groupe : 01

Sigle et titre du cours		
LOG2990 - Projet de logiciel d'application Web		
Professeur	Groupe	Trimestre
Lévis Thériault	Tous	Automne 2021

Jour	Date	Durée	Heures
Vendredi	17 décembre 2021	2h30	13h30 à 16h00

Documentation	Calculatrice	Outils électroniques
☐ Aucune ☒ Toute ☐ Voir directives particulières	☐ Aucune ☒ Toutes ☐ Non programmable (AEP)	**Les appareils électroniques personnels sont interdits.**

Directives particulières

- Le professeur ne répondra à aucune question durant cet examen ; si vous estimez que vous ne pouvez répondre à une question pour diverses raisons, veuillez le justifier puis passez à la question suivante ;
- Il est strictement interdit de débrocher l'examen ;
- IMPORTANT : inscrire votre matricule sur TOUTES les pages numérotées.

Important

Cet examen contient 7 questions sur un total de 13 pages **(excluant cette page).**

La pondération de cet examen est de 25 %

Vous devez répondre sur : ☒ le questionnaire ☐ le cahier ☐ les deux

Vous devez remettre le questionnaire : ☒ oui ☐ non

L'étudiant doit honorer l'engagement pris lors de la signature du code de conduite.

Q1. Application client (5 points)

2.5

Ayant eu énormément de plaisir à développer le projet, vous décidez de le pousser plus loin avec un Sprint 4. Vous êtes responsable d'une nouvelle fonctionnalité permettant de jouer à une partie de la liste des meilleurs scores pour essayer de battre le score obtenu par le gagnant.

Pour ce faire, vous avez décidé de modifier le tableau des meilleurs scores pour y ajouter plus d'informations sur la partie : nom et avatar du gagnant, mode (Classique ou LOG2990) ainsi que le score à battre. Le bouton "Charger" permet de lancer une partie avec les mêmes options que la partie originale. Le bouton "Observer" permet de lancer la partie en mode "observation" permettant de voir le déroulement original de la partie, tour par tour. Cette option est visible seulement si l'attribut "private" est TRUE (ex : la 4e partie de la capture d'écran). Un de vos collègues est déjà responsable pour cette fonctionnalité.

De plus, une barre de recherche permet de filtrer les parties affichées en fonction de leur mode à travers le bouton "Rafraîchir la liste". Si le champ est vide, toutes les parties sont affichées.

Référez-vous à la capture d'écran suivante pour la nouvelle présentation et l'interface finale:

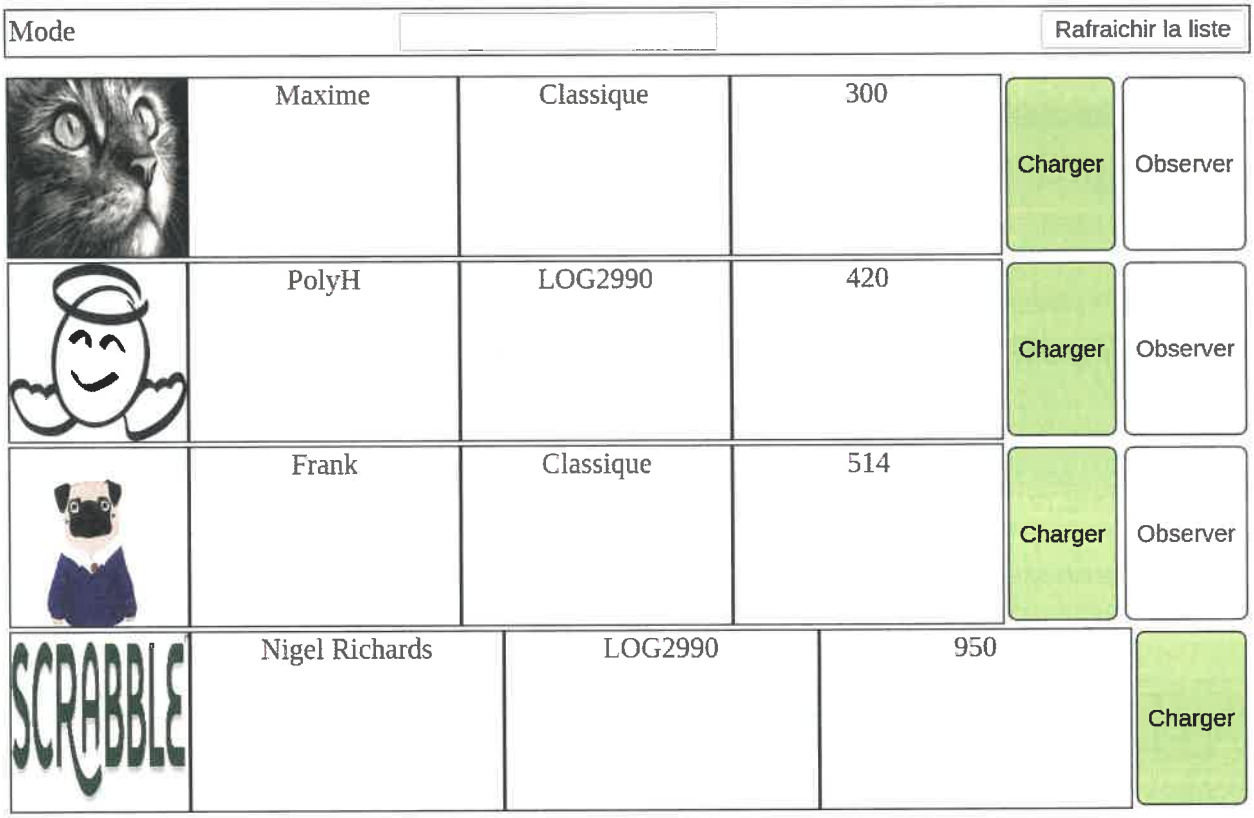


Figure 1 : Liste des meilleurs parties

Voici une partie de l'implémentation de la fonctionnalité que vous devez compléter dans les 3 éléments suivants :***GameDisplayComponent***, ***GameComponent*** et ***CommunicationService***.

Voici l'interface ***Game*** utilisée pour représenter une partie :

```
export interface Game {
   mode: string,
   score: number,
   private: boolean,
   player: {
       name: string,
       avatar: string
   }
}
```

a) **CommunicationService** : la fonction *getGames*() récupère l'information des parties du serveur (assumez une implémentation fonctionnelle). La fonction *getGamesByMode()* permet de récupérer les parties d'un mode spécifique et contient une erreur de **conception**. Quelle est cette erreur? (1 point)

```
export class CommunicationService {
 drawings: Game[];
 constructor() {}
 async getGamesByMode(mode: string): Promise<Game[]> {
   if (!mode) return this.getGames();
   return (await this.getGames()).filter(x => x.mode === mode);
 }          ↳ toPromise → then?
 async getGames(): Promise<Game[]> {...}
}
```

Réponse :

0

Au moment du "return", il manque un "then", qui permet d'attendre le retour de la fonction getGames(). Donc remplacer par :

```
return await this.getGames().then((games: Game[]) => {
        games.filter(x => x.mode === mode),
3);
```

avait déjà là

b) **GameDisplayComponent** : contient la liste des parties à afficher. Chaque partie est représentée par un component séparé (**GameComponent**). Vous pouvez afficher un nombre arbitraire de parties dans la liste. Vous pouvez entrer le nom d'un mode dans le champ de texte et le bouton "Rafraîchir la liste" permet de récupérer les parties de ce mode. Complétez le code du Component et du gabarit HTML directement dans le code fourni. (2 points)

```
export class GameDisplayComponent {
 public mode: string = "";
 public displayedGames: Game[];
 // À COMPLÉTER
 constructor( private communicationService : CommunicationService ) {
   (async () => { this.displayedGames = await this.getGamesByMode(""); })(); }
 private async getGamesByMode(mode: string): Promise<Game[]> {
   return this.communicationService.getGamesByMode(mode);
 }
 async refreshList() { // À COMPLÉTER
        await displayed Games = this. get Games By Mode (mode);
        // on affiche toutes les parties qui sont du bon mode (pas de limite au nombre)
                                                                  this.
 }
}
```

1

Gabarit HTML :

```
<div id="container">
 <!--À COMPLETER-->
 <div id="input-container">
   <label>Mode </label>
   <input id="mode-input" [(ngModel)] = "mode" required />
   <button id="load-button" (click)="refreshList()"> Rafraichir la liste </button>
 </div>
 <div id="game-list">
   <!--À COMPLETER-->
      <div id = "game" *ngFor = " let game Selected of displayed Games" >
        <app-game [game]= "game Selected" />

      </div>

 </div>
</div>
```

c) **GameComponent** : représente la fiche informative de la partie. Chaque fiche contient l'avatar du gagnant, son nom, le mode de la partie, le meilleur score ainsi que 2 boutons qui permettent de lancer ou d'observer la partie. Le bouton "Observer" ne doit pas être visible si la partie est privée. Vous pouvez supposer que les méthodes de lancement sont implémentées et fonctionnelles. Complétez le gabarit HTML directement dans le code fourni. (1.5 points)

```
@Component({
 selector: 'app-game',
 templateUrl: './game.component.html',
 styleUrls: ['./game.component.css']
})
export class GameComponent {
 @Input() game: Game;
 constructor() { }

 loadGame(drawing:Game) {/*...*/}
 watchGame(drawing:Game) {/*...*/}
}
```

Gabarit HTML :

1

```
<div class="parent">
 <!--À COMPLETER-->
 <img id="image" src="{{game.player.avatar}}" />
 <span id="gp">{{game.player.name}}</span>
 <span id="gm">{{game.mode}}</span>
 <span id="gs">{{game.score}}</span>
 <!--À COMPLETER-->
 <button id="load-button" (click)="loadGame()"> Charger </button>
 <button id="w-button" (click)="watchGame(game) *ngIf="!game.private"> Observer </button>
</div>
```

prend un paramètre et n'a pas de valeur par défaut. Oubli?

espace

erreur dans l'énoncé? car visible ssi private == true.

0- dans c) on demande à ce que ce soit cachés si private

cache si private

d) Donnez au moins 1 problème d'expérience utilisateur (UX) dans la capture d'écran fournie. Justifiez brièvement votre réponse. (0.5 point)

0.5

La barre de recherche devrait être résumée à deux boutons radios (2 modes donc un bouton radio par mode). Un utilisateur non-initié pourrait ne pas savoir quoi écrire et ne pas comprendre l'utilité de cette barre.

Q2. Serveur et base de données (3 points)

1.75

a) Quelle est la différence entre le serveur lancé lorsqu'on fait **npm start** dans le dossier **client** et le serveur lancé par la même commande dans le dossier **serveur** de votre projet ? Quel est le rôle de chacun de ses 2 serveurs? **Justifiez** votre réponse. (2 points)

0.75

* Le serveur lancé depuis le client ne permet pas une communication externe au client, contrairement à celui lancé depuis le dossier serveur
* Le serveur lancé depuis client permet de lancer un server mémoire qui est stocké les variables utilisées et d'exécuter des fonctions.
* Le serveur côté serveur permet de faire la communication entre les clients entre-eux et aussi avec la base de données (BD). Il peut également exécuter des fonctions (comme la validation des mots par exemple).

0.5

b) Voici 2 manières différentes de récupérer seulement le nom de tous les dictionnaires sur votre base de données MongoDB. Quelle méthode est à préférer et pourquoi? (1 point)

```
// Option #1
return this.collection.find({}, { projection: { title: 1 } }).toArray();
// Option #2
return (await this.collection.find({}).toArray()).map((x) => {
   return { title: x.title };
});
```

Réponse :

1

Il faut préférer l'option 1 car le traitement de la demande est effectué directement dans la BD (pas besoin d'attendre la réponse pour ensuite faire le traitement localement). La manipulation sur le curseur est toujours plus rapide que si on le transforme en Array et qu'on la manipule localement.

Q3. Assurance Qualité (4 points)

3.5

Voici 2 extraits de code de la *MR* dont la revue de code vous est assignée. Pour chaque extrait, identifiez les problèmes de **qualité du code** et proposez une **amélioration**. Vous pouvez fournir du code en exemple, si vous le trouvez pertinent.

a) La fonction *handleKey* qui gère les touches lors d'un placement de lettres. (2 points)

```
public handleKey(kc: number, i: number, j: number): boolean {
    const letter = String.fromCharCode(kc);
    switch (true) {
        case this.isLetter(kc):
            return this.handleLetter(letter, i, j);
        case this.isBackspace(kc):
            return this.handleOther(i, j);
        case this.isEnter(kc):
            return this.gererEntree(i, j);
        default:
            return false;
    }
}
```

Réponse :

✓ * noms des paramètres → renommer les variables kc, i, j (keyPressed par exemple)

✓ * La fonction gererEntree est en français → renommer fonction (handleEnter()).

✓ * On pourrait sûrement remplacer le switch par une Map mais je ne suis pas certain comment utiliser les Map avec les fonctions.

↳ switch(true) est à proscrire donc plutôt utiliser if

↳ pour les deux premiers faire : if (this.isXXX(keyPressed)) return this.XXX

remplacer par nom des vraies fonctions

↳ pour le dernier : utiliser l'opérateur ternaire,

return this.isEnter(keyPressed) ? this.handleEnter(i,j) : false;

2/2

Bonus : * Il serait sûrement possible d'éviter les fonction isEnter car la vérification est faisable en une ligne. On pourrait alors utiliser un switch ou une Map.

* La fonction n'est pas censée avoir besoin de retourner un boléen, donc soit faire retourner void et simplifier le code, soit renommer la fonction et les autres (has HandledKey, ...)

b) La fonction *replaceLetters* qui modifie le chevalet avec une lettre aléatoire après un échange. Cette fonction contient également une erreur de logique. Quelle est cette erreur? (2 points)

```
replaceLetters(a: Letter): boolean {
    let correct: boolean = isValidItem(a) ? true : false;
    if (correct === true) {
        let newLetter = this.letterBank.obtenirAleatoire(a);
        this.letters.foreach((v, i) => {
            if (this.items[i] === a) { this.items[i] = newLetter; }
        });
        return true;
    }
    else if (correct === false) {
        return false;
    }
}
```

Réponse :

✓ * nom du paramètre : a → letterToReplace

~ * nom de la fonction : replaceLetters → hasReplacedLetter
(boolean, une seule lettre)

✓ * première ligne + "if ... else if" inutiles. Remplacer par : if (!isValidItem) return false;

* newLetter doit être const → let newLetter → const newLetter = ...

✓ * fonction obtenirAleatoire en français + ne devrait pas avoir besoin d'un paramètre
↳ getRandomLetter();

* L'erreur de logique se cache dans l'utilisation du foreach (s'on suppose que newLetter est une lettre aléatoire valide et que obtenirAleatoire est fonctionnelle)

✗ ↳ on ne me rappelle pas par cœur de la doc du foreach (pas sûr qu'il y ait besoin de 2 paramètres mais les fonctions find / findIndex seraient plus adaptées ici.

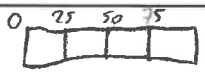
1.5/2

Q4. Tests et Validation (4 points)

2

a) Voici les tests unitaires pour la fonction **calculateCoordinate** de la classe **GridService** qui permet de calculer la coordonnée de départ (coin supérieur gauche) d'une case à partir de l'index et de la taille de la case en pixels. Donnez **l'implémentation de la fonction**. (2 points)

```
it('calculateCoordiante should return 0 for first index', () => {
   const GRID_SIZE = 50;
   const res = service.calculateCoordiante(1, GRID_SIZE);
   expect(res).toBe(0);
});
it('calculateCoordiante should return 150 for index = 3 and size = 75', () => {
   const GRID_SIZE = 75;
   const res = service.calculateCoordiante(3, GRID_SIZE);
   expect(res).toBe(150);
});
it('calculateCoordiante should call sendWarning if index is above 15', () => {
   const GRID_SIZE = 75;
   const spy = spyOn(service, 'sendWarning').and.callThrough();
   const res = service.calculateCoordiante(17, GRID_SIZE);
   expect(spy).toHaveBeenCalled();
   expect(res).toBe(0);
});
```



Réponse :

```
calculate Coordinate ( index : number, gridSize : number ) : number {
    const maxIndex : number = 15;
    let coordinateToReturn : number = 0; // pas obligatoire mais plus lisible
    if ( index > maxIndex ) {
        this. sendWarning();
        return coordinateToReturn;
    }
    coordinateToReturn = (index - 1) * gridSize;
    return coordinateToReturn;

}
```

2/2

b) Voici le constructeur du component **PlayAreaComponent** qui utilise la classe **GridService** ainsi que la configuration de ses tests unitaires. Est-ce que l'utilisation d'un *SpyObj* pourrait être nécessaire dans la configuration? **Justifiez** votre choix. (2 points)

```
constructor(private readonly gridService: GridService) {}

// play-area.component.spec.ts
beforeEach(async () => {
   gridSpy = jasmine.createSpyObj('GridService', ['calculateCoordiante']);
   TestBed.configureTestingModule({
       declarations: [PlayAreaComponent],
       providers: [{ provide: GridService, useValue: gridSpy }],
   }).compileComponents();
});
```

Réponse :

0

Non, je trouve que cela complexifie le code alors qu'on pourrait simplement faire un spy on sur la fonction calculateCoordinate()*. Cela est dû au fait que c'est la seule fonction que l'on souhaite espionner. Si une seconde fonction de ce service devait être espionnée, alors le spyObj deviendrait intéressant.

Ça remplace la dépendance au Service

* et supprime le provider.

Q5. Communication réseau (3 points)

2.25

a) Voici l'implémentation d'une route Express qui permet de récupérer un dictionnaire du serveur. Indiquez au moins deux problèmes avec ce code et proposez une solution possible. (1 point)

```
this.router.post('/getDictionary', (req, res) => {
  const dict = this.dictionaryService.getDictionary(req.query.index);
  res.status(204).send(dict); // code 204 NO CONTENT
});
```

Réponse :

* Il serait préférable de passer l'index du dictionnaire à obtenir directement dans l'URL. Ex: 'get Dictionary/:index'
  ↳ Le paramètre de getDictionary() la ligne suivante serait alors req.params.index

* De plus, le statut de retour devrait être 200 car un contenu est envoyé (le dictionnaire).

0.5/1

b) Vous avez 3 clients qui communiquent par WebSocket avec votre serveur : Client1, Client2 et Client3. Client1 et Client3 font partie de la même *Room* du côté serveur. Client1 vient d'envoyer l'événement "bonjour" avec l'objet **{ data : 1 }** au serveur. Voici la gestion de cet événement du côté serveur. En vous basant sur le code fourni et en supposant que chaque client gère l'événement "allo" dans son propre code, qu'est-ce que recevra chaque client et **pourquoi**? (2 points)

```
socket.on('bonjour', (message) => {
   socket.emit("allo", `Message ${message.data}`);
   socket.broadcast.emit("allo", `Message ${message.data++}`);
});
```

Réponse :

Client 1 recevra : Message 1

Client 2 —— : Message ~~2~~ 1 data++ incrémente après l'envoi

Client 3 —— : message ~~2~~

1.75

JUSTIFICATION. * socket.emit envoie seulement à l'émetteur (celui qui effectue la requête). Or ici c'est Client 1 qui envoie l'événement

* socket.broadcast.emit envoie à tous les clients connectés sauf l'émetteur, donc les Clients 2&3.

Q6. Architecture et Conception (4 points)

2

a) Vous décidez de faire communiquer 2 de vos *Components* à travers un Service partagé. Nommez et justifiez au moins un (1) avantage architectural d'utiliser un Service partagé au lieu de l'utilisation des décorateurs *@Input*/*@Output* pour la communication entre 2 *Components*. (2 points)

0

b) Le concept de *Service* d'Angular se base sur le patron Injection de Dépendances. Donnez un (1) avantage et un (1) désavantage de ce patron. Appuyez vos réponses avec un exemple. (2 points)

2

* Avantage : permet de réutiliser du code pour un même service à travers tout le code. Évite donc la duplication de code. Ex: La fonction isEndGame() de EndGameService peut être utilisé partout tant que le service est injecté.
* Inconvénient : peut causer des dépendances circulaires, fatales au build du projet. Cela arrive lorsqu'un service est injecté à un service qui l'a déjà injecté. Il faut donc prendre cela en compte lors de l'écriture. Ex : service3 injecte service2 (erreur!) service1 injecte

Q7. GitLab et Intégration Continue (2 points)

1,5/2

Lorsqu'on crée une demande fusion (*Merge Request*) sur GitLab, il est possible de bloquer la fusion tant qu'une pipeline qui lui est associée n'est pas exécutée au complet sans erreurs avec l'option **"Pipelines must succeed".** Donnez un (1) **avantage** et un (1) **désavantage** de l'utilisation de cette option sur votre processus de travail. **Justifiez** brièvement votre réponse et dites quel serait votre choix de configuration.

Avantage : Permet de garder du code qui compile, qui est "propre" (passe le lint) et qui valide les tests sur les branches. En cas de problème, l'erreur est assurée d'être locale non sur GitLab.

-0,5 erreur est sur gitlab. La MR rouge sur gitlabed

Désavantage : Ce processus peut prendre une dizaine de minutes à chaque push. Cela peut s'avérer être long si les changements n'ont pas d'impact sur le fonctionnement (ex: hotfix des couleurs des boutons sur la page d'accueil).

+1

Je choisirais tout de même d'utiliser cette option dans ma configuration car son avantage est fondamental (le code partagé est toujours compilable et testé) à un projet de groupe. Pour l'inconvénient, il existe un bot qui permet de "merge" automatiquement le code lorsque le pipeline réussit. Cela peut faire gagner du temps et amortir le désavantage qui reste minime comparé à ce que cette option apporte.

✓